Seba Sohayok (■■■■ ■■■■■■) — Master Study Note

Complete Zero-to-Hero Architectural, Algorithmic & Viva Defense Manual

Course: CSE 3202 (Artificial Intelligence Sessional) | **Domain:** Bangladesh Public Administrative Services | **Architecture:** Goal-Based AI Agent + Hybrid RAG + Zero Hallucination

Chapter 1: The Problem & System Motivation (Why This Project Exists)

Bangladeshi citizens frequently encounter severe friction when navigating public administrative procedures (specifically e-Passport and BRTA Driving Licence services). Lack of clear information leads to intermediary exploitation (dalals), visits to wrong regional offices, or payment of incorrect government fees. Standard conversational AI models (such as direct ChatGPT or raw Gemini API) fail catastrophically in this domain because of **Factual Hallucination**—inventing non-existent fees, outdated document requirements, and fictional procedures. **Seba Sohayok** enforces **Zero-Fact Hallucination** by strictly divorcing conversational dialogue from factual data generation.

Rule 1: Zero Hallucination Gemini is strictly prohibited from generating fees, deadlines, or rules. All facts are served verbatim from human-verified official datasets.	**Rule 2: Bimodal Execution** Direct action chip clicks bypass the LLM completely, resolving in **~5-15ms**. Natural text queries pass through the cognitive RAG engine.	**Rule 3: Confidence Refusal** Queries scoring below confidence threshold (**0.62**) trigger an explicit refusal card rather than ungrounded speculation.

Chapter 2: System Architecture & Execution Lifecycle

The system is partitioned into four clear operational layers:

- **Client Presentation Layer (web/):** Glassmorphic responsive console featuring Claude-style live thinking animation, expandable reasoning summary timeline, progressive word-by-word streaming with blinking cursor, and viewport anchoring preventing empty scroll voids.
- **API Gateway & Edge (app/main.py, app/limits.py):** Asynchronous ASGI application powered by FastAPI and Uvicorn. Implements in-memory sliding-window token bucket rate limiting (20 req/min, 200 req/day per IP).
- **Goal-Based Agent Engine (app/agent.py):** Maintains conversational state tracking, regex guardrails intercepting Out-of-Scope (land/NID/tax) queries and Smalltalk greetings, and triggers clarification loops when user goals are ambiguous.
- **Hybrid Retrieval Layer (app/retriever.py, data/):** In-memory normalized vector matrix (data/embeddings.npz) performing ultra-fast vectorized NumPy cosine similarity plus lexical keyword boosting.

Bimodal Request Routing: Fast-Path vs. Cognitive Chat Path

1. **Fast Path (POST /api/action):** Triggered when citizens click pre-structured action chips (e.g., '■■■■■■■■ ■■ ■■?'). Bypasses Gemini entirely, reading verified records directly from memory in **< 15ms** with zero API cost.
2. **Cognitive Chat Path (POST /api/chat):** Triggered by free-text. Gemini extracts intent/slots, generates query embedding, scans the NumPy matrix, gates by threshold (0.62), and stitches 1 connecting sentence with verbatim facts.

Chapter 3: Deep Dive into RAG (Retrieval-Augmented Generation)

RAG fundamentally changes how AI works: instead of relying on memory, the AI is given the exact official government document to consult before speaking. Our pipeline executes in 5 mathematical stages:

Stage 1: Semantic Chunking: Knowledge entries are chunked semantically by functional service units: (Domain, Subtype, Field). Each chunk contains: ID, title, fee/steps table, official source URL, verification date, and multilingual keywords.

Stage 2: 3072-Dimensional Dense Vectors: Each entry is embedded into a 3072-d float32 vector using gemini-embedding-001 and L2-normalized (||e_d|| = 1). Saved in data/embeddings.npz.

Stage 3: Hybrid Scoring Formula: Dense semantic similarity is combined with sparse lexical keyword matching:

FinalScore(q, d) = CosineSimilarity(e_q, e_d) + $\beta \cdot$ **KeywordBoost(q, d)**
where CosineSimilarity(e_q, e_d) = e_q · e_d (since ||e_q|| = ||e_d|| = 1)
- **e_q:** Normalized 3072-d user query vector
- **e_d:** Pre-computed entry vector in embeddings.npz
- **KeywordBoost(q, d):** Exact match score covering Bangla, English, and phonetic Banglish
- β: Term frequency heuristic weight (0.15) boosting exact administrative matches

Stage 4: Why In-Memory NumPy Over ChromaDB / Pinecone? For ~70 atomic, high-accuracy public service records, external vector DBs add cold-start network lag and disk bloat. A vectorized NumPy dot product across 70 vectors runs in **0.3 milliseconds** directly in RAM with zero external dependencies.

Stage 5: Confidence Gating: If FinalScore < 0.62, the query is deemed out-of-scope, triggering a safe refusal card. If ≥ 0.62, the top record is returned with verified source URL.

Chapter 4: Complete File-by-File Technical Directory

File	Layer	Technical Responsibilities & Code Invariants
app/main.py	Gateway	FastAPI app entry, routes (/api/chat, /api/action, /api/meta, /api/feedback, /health), static mounting, Admin Basic Auth.
app/agent.py	Agent Core	State tracking, regex guardrails (intercepts Out-of-Scope land/tax/NID queries and Smalltalk greetings), clarification loop.
app/retriever.py	Search	In-memory NumPy matrix cosine scanning, keyword boosting, threshold filtering (0.62), record fetching.
app/index.py	Indexer	Batch ingestion utility. Reads JSONs, queries Gemini Embedding API, normalizes vectors, saves embeddings.npz.
app/kb.py	Knowledge	Validates 14 subtypes & 5 fields. Loads and structures JSON datasets in runtime memory.
app/llm.py	GenAI SDK	Google google-genai wrapper, structured JSON output mode, transient error retries, embedding generation.
app/prompts.py	Prompts	Few-shot intent extraction prompts, slot filling rules, and zero-hallucination boundaries.
app/limits.py	Rate Limit	In-memory sliding-window token bucket per IP (20 req/min, 200 req/day). Prevents quota exhaustion.
app/session.py	State	In-memory multi-turn session store: active domain, resolved subtype, extracted slots, client language.
app/analytics.py	Telemetry	SQLite database (data/analytics.db) logging queries, agent decisions, turn latency, and feedback thumbs.
data/*.json	Verified KB	Human-verified datasets for e-Passport and BRTA Driving Licences (fees, rules, documents, delivery times).
web/index.html	Client UI	Claude live thinking animation, expandable reasoning summary timeline, progressive streaming, viewport anchoring.
run_live.py/.bat	Harness	Port 8000 auto-freeing utility, Uvicorn ASGI launcher, and automated Cloudflare Tunnel manager.

Chapter 5: Detailed Request Lifecycle (From Input to UI Render)

Example: Citizen types '[redacted]?':

1. **T+0ms (Client):** Query sent via POST /api/chat. Live thinking animation begins with coral pulsing dots (#D9663D).
2. **T+15ms (Gateway):** limits.py checks IP quota token bucket; passes query.
3. **T+30ms (Agent Guardrails):** Regex verifies query is within scope (not land, NID, or tax) and not smalltalk.
4. **T+350ms (LLM Intent Parsing):** Gemini 2.5 Flash extracts JSON: {domain: 'passport', subtype: 'urgent', field: 'fees'}.
5. **T+700ms (Embedding):** Query converted to 3072-d normalized vector via gemini-embedding-001.
6. **T+750ms (Hybrid Retrieval):** NumPy dot product across embeddings.npz yields Confidence = 0.96 (Passes 0.62 threshold).
7. **T+1200ms (Surface Synthesis):** Gemini authors 1 introductory sentence; official facts card attached verbatim from passport.json.
8. **T+1400ms (Progressive UI Stream):** Thought bar locks at top; words stream in word-by-word with blinking cursor (45-58ms/word); sections glide down progressively; scrollToTopOf keeps the answer cleanly in view without empty voids.

Chapter 6: Viva Defense Guide (How to Impress Course Teacher)

Key questions your course teacher will ask, and the exact winning answers:

Q1: Why didn't you just use standard ChatGPT or raw Gemini API?

Raw foundation LLMs hallucinate facts. In public administrative services, quoting an incorrect fee or document causes harassment for citizens. Our architecture guarantees Zero-Fact Hallucination by decoupling conversational understanding from factual data retrieval.

Q2: Why did you build custom RAG instead of using LangChain or LlamaIndex?

LangChain adds heavy abstraction overhead and packaging bloat. By implementing our own NumPy-accelerated vector search, slot extraction, and token-bucket rate limiting, our system runs in < 1ms retrieval latency with 100% architectural transparency and zero black-box dependencies.

Q3: What makes this a 'Goal-Based Agent' (Russell & Norvig model)?

A simple reflex chatbot provides fixed, single-turn responses without memory. Seba Sohayok tracks an internal session state (domain, subtype, slots). When user queries are ambiguous ('[redacted]'), the agent detects the missing goal parameter and conducts an interactive clarification loop until the goal is fully resolved.

Q4: How do you handle Banglish or mixed queries ('urgent passport fee koto')?

First, Gemini-embedding-001 is natively trained on massive multilingual and transliterated corpora. Second, our Hybrid Retrieval algorithm applies a Lexical Keyword Boost across Bangla, English, and phonetic Banglish keyword sets embedded in each record.

Q5: How does your UI achieve a Claude-style progressive presentation?

We engineered terracotta coral pulsing dots with live thinking progression, an expandable reasoning timeline, word-by-word streaming with cursor tracking (45-58ms/word), step-by-step section reveals (260ms/step), and viewport anchoring (scrollToTopOf) preventing blank scroll voids.